

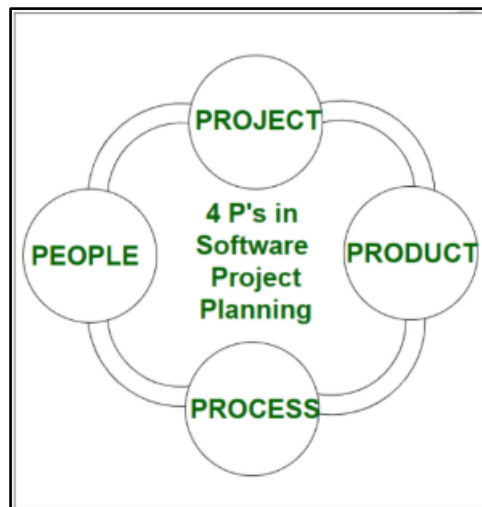
STE Chapter 4 Assignment 2

1. Explain the Management Spectrum and the 4P's (People, Product, Process and Project) of Software Project Management.

Ans.

These components play a very important role in your project that can help your team meet its goals and objective.

The Management Spectrum for software projects centers on four essential components - People, Product, Process, and Project.



1. People:

The most important component of a product and its successful implementation is human resources. In building a proper product, a well-managed team with clear-cut roles defined for each person/team will lead to the success of the product. We need to have a good team in order to save our time, cost, and effort. Some assigned roles in software project planning are project manager, team leaders, stakeholders, analysts, and other IT professionals. Managing people successfully is a tricky process which a good project manager can do.

2. Product

As the name inferred, this is the deliverable or the result of the project. The project manager should clearly define the product scope to ensure a successful result, control the team members, as well technical hurdles that he or she may encounter during the building of a product. The product can consist of both tangible or intangible such as shifting the company to a new place or getting a new software in a company.

3. Process

In every planning, a clearly defined process is the key to the success of any product. It regulates how the team will go about its development in the respective time period. The Process has several steps involved like, documentation phase, implementation phase, deployment phase, and interaction phase.

4. Project

The last and final P in software project planning is Project. It can also be considered as a blueprint of process. In this phase, the project manager plays a critical role. They are responsible to guide the team members to achieve the project's target and objectives, helping & assisting them with issues, checking on cost and budget, and making sure that the project stays on track with the given deadlines.

2. Explain LoC (Line of Code) and Function Point (FP) as software size estimation metrics. Differentiate between LoC and FP.

Ans.

Lines of Code (LOC) :

Measures the total number of lines in the source code, which can be quantified as KLOC (thousands of lines of code).

Function Points (FP) :

Quantifies the complexity of the software based on the number of functions, inputs, outputs, files, and interfaces it provides to the user. This metric is language independent and focuses on the functionality offered to the user.

Difference Between LoC and FP

LoC (Lines of Code)	FP (Function Points)
1. Measures software size by counting the number of lines of code.	1. Measures software size based on the functionality delivered to the user.
2. It is a language-dependent measurement.	2. It is a language-independent measurement.
3. It can be calculated after coding.	3. It can be estimated during the requirement analysis phase.
4. It depends on the programming language used.	4. It depends on inputs, outputs, files, interfaces and user queries.
5. It is easy to count but difficult to estimate before development.	5. It requires detailed analysis but helps estimate effort early.
6. Example: A program contains 5,000 lines of code.	6. Example: A software system has 100 Function Points.

3. Explain the Heuristic, Analytical and Empirical approaches to Software Project Cost Estimation.

Ans.

1. Heuristic Approach

- It uses experience, rules of thumb and mathematical models to estimate software project cost.
- It is based on historical data and expert judgment.
- Example: COCOMO model, which estimates effort based on the size of the software.

2. Analytical Approach

- It uses mathematical formulas and logical analysis to estimate project cost.
- It breaks the project into smaller tasks and calculates the effort required for each task.
- The total cost is estimated by combining the effort of all tasks.
- Example: Estimating cost using algorithmic complexity and resource calculations.

3. Empirical Approach

- It uses past project data, observations and actual experience to estimate the cost of a new project.
- It compares the new project with previously completed projects of similar size and complexity.
- The estimate is based on actual results and collected data.
- Example: Estimating the cost of a new banking application using the cost of a previously developed banking application.

4. Explain COCOMO (Constructive Cost Model) and COCOMO II. Differentiate between COCOMO and COCOMO II.

Ans.

1. COCOMO (Constructive Cost Model)

COCOMO is a software cost estimation model developed by Barry Boehm. It is used to estimate the effort, development time and cost required to develop a software project based on its size in KLOC (Thousands of Lines of Code).

COCOMO has three modes:

- Organic: Small and simple projects developed by experienced teams.
- Semi-detached: Medium-sized projects with moderate complexity.
- Embedded: Complex projects with strict hardware, software or operational constraints.

Basic COCOMO formula:

- Effort (Person-Months) = $a \times (KLOC)^b$
- Development Time = $c \times (Effort)^d$

Where a, b, c, d are constants depending on the project mode.

2. COCOMO II

COCOMO II is an updated version of COCOMO designed for modern software development practices, such as object-oriented development, reuse of software and iterative development.

It estimates effort using software size, scale factors, cost drivers and project characteristics.

COCOMO II has three sub-models :

- Application Composition Model: Used for projects developed using reusable components and rapid application development tools.
- Early Design Model: Used to estimate effort during the early stages of software design.
- Post-Architecture Model: Used when the software architecture is defined and detailed information is available.

COCOMO II formula:

$$\text{Effort} = A \times (\text{Size})^E \times \prod EM$$

Where:

- A = Constant
- Size = Software size
- E = Exponent based on scale factors
- EM = Effort multipliers

3. Difference Between COCOMO and COCOMO II

COCOMO	COCOMO II
1. Developed for traditional software development.	1. Designed for modern software development.
2. Based mainly on KLOC.	2. Supports object points, function points and KLOC.
3. Has Basic, Intermediate and Detailed models.	3. Has Application Composition, Early Design and Post-Architecture models.
4. Less suitable for software reuse.	4. Supports software reuse and component-based development.
5. Suitable for sequential development approaches.	5. Supports iterative and incremental development.
6. Uses fewer project factors.	6. Uses scale factors and effort multipliers.
7. Less flexible for modern projects.	7. More flexible and suitable for modern projects.

5. Prepare an RMMM (Risk Mitigation, Monitoring and Management) Plan for a suitable Software Project.

Ans.

RMMM Plan for a Student Management System

RMMM (Risk Mitigation, Monitoring and Management) is a plan used to identify, analyze and control risks that may affect a software project.

Project: Student Management System

1. Risk Mitigation, Monitoring and Management Plan

Risk	Mitigation	Monitoring	Management
1. Requirement changes	Gather clear requirements before development.	Review requirements regularly.	Update the design and schedule.
2. Technical failure	Use tested technologies and maintain backups.	Monitor system performance.	Restore backups and fix technical issues.
3. Schedule delay	Prepare a proper project schedule.	Track daily progress.	Reassign tasks and adjust deadlines.
4. Data loss	Perform regular database backups.	Check backup status.	Restore data from backups.
5. Security threats	Use authentication and access control.	Monitor login activities.	Block unauthorized access and fix vulnerabilities.
6. Lack of resources	Allocate sufficient resources and skilled developers.	Review resource availability.	Arrange additional resources or redistribute tasks.

2. RMMM Activities

- Risk Mitigation: Actions taken to reduce the probability or impact of risks before they occur.
- Risk Monitoring: Regularly tracking identified risks and detecting new risks during development.
- Risk Management: Taking corrective actions when a risk occurs to minimize its impact on the project.